

Python Common Services Transport Layer

Table of Contents

Copyright Notice	3
Change History	4
1. Introduction	5
2. Installation	6
3. Implementing a Transport layer	7
3.1. Transport Protocol	7
3.1.1. identity property	7
3.1.2. session context manager	7
3.1.3. subscribe function	7
3.1.4. send_message function	8
3.1.5. default_security attribute	8
3.1.6. file_transfer_service attribute	8
3.1.7. stream_service attribute	8
3.2. FileTransferService	9
3.2.1. start_transfer function	9
3.2.2. FileTransfer class	9
stop function	9
3.3. StreamService	9
3.3.1. start_stream function	10
3.3.2. StreamTask class	10
stop function	10
3.3.3. Source Streams	10
UDP Source Streams	10
3.3.4. Destination Streams	11
3.3.5. UDP Destination Streams	11
4. Using a Transport Layer	12
4.1. Transport class	12
4.1.1. identity property	12
4.1.2. session context manager	12
4.1.3. send_message function	12
4.1.4. subscribe function	13
4.1.5. default_security attribute	13
4.2. File Transfers	13
4.2.1. Tasker	13

4.2.2. Source Transfers	14
Static Sources	15
Custom Sources	15
4.2.3. Destination Transfers	16
Static Destinations	17
Custom Destinations	17
4.3. Streams	17
4.3.1. Tasker	18
4.3.2. Source Streams	19
RTP Source Streams	19
4.3.3. Destination Streams	19

Copyright Notice

© 2019-2023 – CROWN OWNED COPYRIGHT. All Rights Reserved.

The copyright of this Software is vested in the Crown and the Software is the property of the Crown.

This Software is released in confidence, to the recipients nominated by the Crown and is for use only for the purposes of the Crown.

This Software shall not be copied or further disclosed except as authorised by the Crown. Any further disclosure shall be under the terms of this legend and any copies shall be the property of the Crown. On completion of the work in connection with which this Software is released the recipient shall return the Software (and any correspondence) to the issuing authority)

Change History

1.0	Initial Version
2.0	Allow Streams to use protocols other than UDP

1. Introduction

The PyCS framework allows Python application to connect to the Common Services network. The Pycs transport layer provides for message routing and delivery, along with file transfers and UDP/RTP streaming. The `pycs.transport` package provides an abstract transport layer that allows for different underlying transport protocols to be used.

The `pycs.transport.Transport` class provides a python Protocol that any concrete transport implementations must provide. This protocol has a very minimal interface, providing the ability to send and receive messages, and monitor the connection state.

File transfers are handled by the `pycs.transport.FileTransferService` abstract base class. This class provides the ability to register files and directories as sources and destinations for file transfer requests, expect and request transfers, and initiate a transfer. Concrete implementations must provide functionality to initiate a transfer on the network, monitor for transfers requested by other network entities, and make the actual file transfer.

Streams are handled by the `'pycs.transport.StreamService'` abstract base class. This class provides the ability to register sources and destinations for stream requests, expect and request streams, and initiate a stream. Concrete implementations must provide functionality to initiate a stream from this service, monitor for streams requested by other hubs, and start and stop streams.

The Stream Service also provides a number of helper classes for running UDP and RTP source and destination streams.

2. Installation

pys requires an installation of python at least version 3.13. The use of pyenv and virtual environments is recommended.

```
$ python -m venv venv
$ ./venv/bin/pip install pys.transport
```

To use the pys package interactively, run python inside the virtual environment

```
$ ./venv/bin/python
>>> import pys.transport
>>> help(pys.transport)
Help on package pys.transport:
...
```

3. Implementing a Transport layer

3.1. Transport Protocol

The `pys.transport.Transport` Protocol describes the functionality required of a Common Services message transport layer. Classes implementing this protocol provide the ability to send messages to the Common Services network, and subscribe to receive messages.

A concrete implementation of the protocol must implement the following functionality.

3.1.1. `identity` property

```
@property
def identity(self) -> str | None:
```

The `identity` property provides clients with the identity of the current session that may be used by other network entities to send messages to this transport. If no current connection session is active, this must have a value of `None`. This identity is likely provided by the network upon successful connection.

3.1.2. `session` context manager

```
@asynccontextmanager
async def session(self) -> AsyncGenerator[str]:
```

The `session` context manager allows clients to monitor when the transport connects and disconnects. An implementation must yield the identity of the current session when a connection is made, and enter the context. When the connection disconnects, the context must exit.

3.1.3. `subscribe` function

```
async def subscribe(
    self,
    tag: str,
) -> AsyncGenerator[tuple[Xml, CorrelationId, Source]]:
```

The `subscribe` function allows clients to subscribe to messages received with a particular XML element tag. When the transport receives messages with a matching tag, this function yields a tuple containing the XML message, a correlation id and a source id.

A typical implementation might create a queue which the transport can push messages onto, and loop over this queue yielding messages.

```
async def subscribe(
    self,
    tag: str,
) -> AsyncGenerator[tuple[Xml, CorrelationId, Source]]:
    queue = self.add_queue(tag, queue)
    try:
        while True:
            yield await queue.get()
            queue.task_done()
    except asyncio.QueueShutDown:
        pass
    finally:
        self.remove_queue(tag, queue)
```

3.1.4. **send_message** function

```
async def send_message(
    self,
    destination: str,
    payload: Xml,
    *,
    security: Security | None = None,
    correlation_id: CorrelationId = None,
):
```

The **send_message** function allows clients to send a message to recipients on the network. If no security marking is supplied, the transport should apply the markings stored in the **default_security** attribute. Any **correlation_id** supplied by the client should be applied to the message to allow other network clients to match up requests and replies.

3.1.5. **default_security** attribute

This attribute should be populated with a default security marking to be applied to any messages which do not supply a more specific marking.

3.1.6. **file_transfer_service** attribute

This attribute should be populated with a **FileTransferService** instance, and is used by clients to manage file transfers between network entities.

3.1.7. **stream_service** attribute

This attribute should be populated with a **StreamService** instance, and is used by clients to manage file transfers between network entities.

3.2. FileTransferService

The `pys.transport.FileTransferService` abstract base class provides much of the functionality required to perform file transfers.

A concrete implementation must implement the `start_transfer` abstract function to request that the network initiate a file transfer between the endpoints identified.

Additionally, the service should monitor the network for any external requests to start incoming or outgoing file transfers to or from this endpoint, and initiate source or destination file transfers over an appropriate mechanism, for example HTTP, FTP or carrier pigeon.

3.2.1. start_transfer function

```
@abstractmethod
async def start_transfer(
    self, source: str, destination: list[str], reference_id: str
) -> FileTransfer:
```

The `start_transfer` function should make a request to the network to initiate a file transfer between the supplied endpoints. It must return a concrete `pys.transport.FileTransfer` object which can be used by clients to monitor the progress of the transfer.

3.2.2. FileTransfer class

The `pys.transport.FileTransfer` abstract base class is used by clients to monitor and control the progress of file transfers. A concrete implementation must implement the `stop` function to request the network to stop the in-progress transfer. It must also call `update_status` with a `pys.transport.FileTransferStatus` object as the transfer progresses to notify clients of progress.

stop function

```
@abstractmethod
async def stop(self) -> None:
```

The stop function should send a request to the network to cancel the transfer.

3.3. StreamService

The `StreamService` abstract base class provides much of the functionality required to perform UDP and RTP streaming.

A concrete implementation must implement the `start_stream` abstract function to request that the network initiate a stream between the endpoints identified.

Additionally, the service should monitor the network for any external requests to start incoming or outgoing streams to or from this endpoint, and initiate source or destination streams as

appropriate.

3.3.1. `start_stream` function

```
@abstractmethod
async def start_stream(
    self, source: str, destination: list[str], reference_id: str, passive: bool
) -> StreamTask:
```

The `start_stream` function should make a request to the network to initiate a file transfer between the supplied endpoints. It must return a concrete `StreamTask` object which can be used by clients to monitor the progress of the transfer.

3.3.2. `StreamTask` class

The `StreamTask` abstract base class is used by clients to monitor and control the progress of streams. A concrete implementation must implement the `stop` function to request the network to stop the in-progress stream. It must also call `update_status` as the stream progresses to notify clients of progress.

`stop` function

```
@abstractmethod
async def stop(self) -> None:
```

The `stop` function should send a request to the network to cancel the stream.

3.3.3. Source Streams

When a request is received to start a local source stream, the transport implementation should initiate the `SourceStream` instance in an async context manager, and call the `'run_stream()'` function, passing a `PacketSink` implementation to forward the data packets via whatever mechanism is in use.

```
class ExamplePacketSink(PacketSink):
    async def send_packet(self, data: bytes):
        print(f"sending packet length {len(data)}")

    async def start_stream(stream: SourceStream):
        async with stream, ExamplePacketSink() as sink:
            await stream.run_stream(sink)
```

UDP Source Streams

A common mechanism to send packet streams is via the UDP protocol, and so a `UdpPacketSink` class is provided to send packets to the supplied host and port over UDP.

```

async def start_stream(stream: SourceStream):
    async with stream, UdpPacketSink('localhost',12345) as sink:
        await stream.run_stream(sink)

```

3.3.4. Destination Streams

When a request is received to start a local destination stream, the transport implementation should initiate the `DestinationStream` instance in an async context manager, and call the `'run_stream()'` function, passing a `PacketSource` implementation to forward the data packets via whatever mechanism is in use.

```

class ExamplePacketSource(PacketSource):
    async def packets(self) -> AsyncGenerator[bytes]:
        yield b'hello world'

async def start_stream(stream: DestinationStream):
    async with stream, ExamplePacketSource() as source:
        await stream.run_stream(source)

```

3.3.5. UDP Destination Streams

A common mechanism to receive packet streams is via the UDP protocol, with multiple streams multiplexed over a single UDP socket, so `UdpPacketSource` and `UdpDemuxer` classes are provided to receive packets from a UDP socket and de-multiplex them to individual `DestinationStream` instances.

The `UDPDemuxer` class listens for UDP packets on a socket and de-multiplexes them based on a packet identifier extracted from the data by a bit mask, passing on to any `UdpPacketSources` registered against that identifier. Each `UDPDemuxer` can only demux streams with the same packet identifier mask. By default the demuxer will bind to an ephemeral port on any network interface, but an explicit port and bind address may be supplied if required.

A `udp://` url is available via the `url` property on the demuxer to enable the destination to be advertised to sources.

```

async def start_stream(stream: DestinationStream, demuxer: UdpDemuxer, packet_id: int):
    async with stream, UdpPacketSource(demuxer,packet_id) as source:
        await stream.run_stream(source)

async with UdpDemuxer(PacketIdentifierMask(offset=0,mask=0xff), port=12345) as demuxer:
    print('Listening on {demuxer.url}')
    await start_stream(ExampleDestinationStream(), demuxer, 42)

```

4. Using a Transport Layer

4.1. **Transport** class

The **Transport** class is an abstract base class, and an implementation must be provided for the transport mechanism in use. See the documentation for your implementation of choice for details of how to create a new instance of a transport class and set it running.

4.1.1. **identity** property

```
@property
def identity(self) -> str | None:
```

The **identity** property supplies the current network identity of the connection. If no connection is currently active **None** will be returned.

4.1.2. **session** context manager

```
@asynccontextmanager
async def session(self) -> AsyncGenerator[str]:
```

The **session** async context manager waits for a transport connection to be established on entry, and then yields the network identity of the connection. On exit it waits for the connection to be lost. This enables a client to monitor the connection status and perform any necessary operations on connection or disconnection.

Typical session usage scenario

```
while connection_wanted:
    async with transport.session() as identity:
        print(f"Connected as {identity}")
        setup_connection(identity)
    print("Disconnected")
    cleanup_connection()
```

4.1.3. **send_message** function

```
async def send_message(
    self,
    destination: str,
    payload: Xml,
    *,
    security: Security | None = None,
    correlation_id: CorrelationId = None,
```

```
):
```

The `send_message` function enables clients to send messages onto the network. The `destination` is the network address of the intended recipient. The `payload` is an lxml XML element. An explicit `security` marking may be provided, with the `default_security` attribute being used if none is supplied. The `correlation_id` may be used by the wider network to allow replies to be identified.

4.1.4. `subscribe` function

To receive messages from the network, the `subscribe` async generator allows clients to subscribe to messages with a particular XML tag. The tag should be supplied in `lxml format`, that is with the full namespace in curly braces before the tag, for example `{urn:csmf:message:mfdoc}MfDoc`.

The generator will yield a tuple for each message received, consisting of the XML payload, any correlation id supplied by the network, and the network identity of the sender.

The subscription will be cancelled when the async generator exits.

```
async def example(transport: Transport):
    async for document, correlation_id, sender in transport.subscribe('{urn:csmf:message:mfdoc}MfDoc'):
        print(f"Received document from {sender}:\n{lxml.etree.tostring(document, pretty_print=True)}")
```

4.1.5. `default_security` attribute

The `default_security` attribute may be used to set the security marking on messages sent over the network that do not supply a superseding marking on the call to `send_message`.

4.2. File Transfers

A transport implementation is able to broker file transfers between two other network clients or between another client and itself. We will define three roles partaking in a file transfer, the 'Tasker' which instigates the transfer, the 'Source' which provides the file to be transferred, and the 'Destination' which receives the file. The transport instance can fulfill all these roles. The service handling these requests is accessed via the `file_transfer_service` attribute.

4.2.1. Tasker

As the Tasker, the transport can request files to be transferred from one client to another (where either or both of these can be itself). The source and destination ids of the file to be transferred should be communicated to the application by some other means. A `FileTransfer` object is returned from each request, and status updates are provided from the `subscribe_status` generator on the transfer. The status generator will exit when the transfer is completed.

```
from pycs.transport import Transport
import asyncio

async def example(transport : Transport ):
```

```
# Request a transfer from client 12345 to client 54321
transfer = await client.file_transfer_service.request_transfer(
    source='example:client:12345#a_file',
    destination=['example:client:54321']
)

async for status in transfer.subscribe_status():
    print(f"File transfer {status.percent_complete}% complete")
```

The client can also request a file to be transferred to itself and placed in a specific file location by means of the `request_file` function. In this instance the transport is serving as both Tasker and Destination. If a string is supplied as the destination, this will be converted into a `FilesystemDestinationFile` in which to place the received file. Otherwise a `DestinationFile` object may be supplied to receive the file. The `transfer.result` attribute holds the `DestinationFile` instance, and `await`ing this instance will wait for the transfer to complete.

```
async def example(transport : Transport ):
    # Request a transfer from client 12345 and place the result in /tmp/a_file
    transfer = await transport.file_transfer_service.request_file(
        source='example:client:12345#a_file',
        destination_file='/tmp/a_file'
    )

    await transfer.result
    print(f"File {transfer.result.filename} received")

    transfer2 = await transport.file_transfer_service.request_file(
        source='example:client:12345#a_file',
        destination_file=StaticDestinationFile()
    )

    await transfer2.result
    data = await transfer2.result.data
    print(f"{len(data)} bytes received")
```

4.2.2. Source Transfers

The network may request files to be transferred from a local Transport. These requests will be automatically handled by the transport instance.

A client may register individual files against an id using `register_file`. Once registered, these files may be requested by another client using a uri of the form `<client id>#<file id>` as the source URI in the request, where `<client id>` is the network identifier of the source client. Once the file is no longer available it may be deregistered using `deregister_file`.

```
def example(transport: Transport):
    transport.file_transfer_service.register_file(id='a_file', filename='/home/files/a_file.txt')
```

```
...
transport.file_transfer_service.deregister_file('a_file')
```

Alternatively, a client may register a directory against an id. Once registered, files within this directory may be requested by another client using a URI of the form `<client id>#<dir id>/<path to file>`, where `<path to file>` is the path of the file relative to the directory that was registered against the id. Once the directory is no longer available it may be deregistered using `FileTransfer.deregister_file`.

```
def example(transport: Transport):
    transport.file_transfer_service.register_file( id='files', filename='/home/files')
    ...
    transport.file_transfer_service.deregister_file('files')
```

If multiple ids are registered with overlapping paths, the most specific path will be used. So, given

```
def example(transport: Transport):
    transport.file_transfer_service.register_file('files', '/home/files')
    transport.file_transfer_service.register_file('files/cuts/audio', '/home/audio')
    transport.file_transfer_service.register_file('files/cuts/audio/bring_me_sunshine.mp3', '/music/sunshine.mp3')
```

requesting `files/cuts/audio/bring_me_sunshine.mp3` would transfer `/music/sunshine.mp3`, whereas requesting `files/cuts/audio/im_a_lumberjack.mp3` would transfer `/home/audio/im_a_lumberjack.mp3`, and `files/cuts/metadata/today.dat` would transfer `/home/files/cuts/metadata/today.dat`.

Any transfer requests for files which do not exist or for ids which have not been registered will be rejected.

Static Sources

Rather than reading files from the filesystem, a `StaticSourceFileFactory` can be registered as a source rather than a directory or file path. This enables in-memory data sources to be accessed.

```
def example(transport: Transport):
    factory = StaticSourceFileFactory(security=client.default_security)
    factory.add_file('hello/world', b'Hello World!')

    transport.file_transfer_service.register_source('static_data', factory)
```

Custom Sources

Other custom source factories can also be created by implementing the `SourceFileFactory` protocol. The `call` function on this class must take an `item_id` parameter and return an object conforming to the `SourceFile` protocol.

```
class CustomSource(SourceFile):
    def __init__(self):
```

```

        self.cache_ok = True
        self.security = Security()

    async def md5sum(self) -> str: return "5eb63bbbe01eeed093cb22bb8f5acdc3"

    @asynccontextmanager
    async def data(self) -> AsyncGenerator[bytes]:
        yield b"hello world"

class CustomSourceFactory(SourceFileFactory[CustomSource]):
    def __call__(self, item_id: str = "") -> CustomSource|None:
        return CustomSource()

```

4.2.3. Destination Transfers

The **Transport** can act as a destination for transfers by requesting a transfer to itself and supplying a filename for the received file, as shown in the Tasker examples above.

A Transport can also act as a destination for transfers instigated by another Tasker by calling **FileTransfer.expect_file** with the reference id of the transfer and a filename in which to put the file.

```

def example(transport: Transport):
    transport.file_transfer_service.expect_file('transfer-123', '/home/files/file-123.dat')

```

Alternatively, a client may register a directory against an id. Once registered, any transfers with a source URI of the form **<client id>#<dir id>/<path to file>**, will be placed in **<path to file>** is the path of the file relative to the directory that was registered against the id.

Any files received in this manner will be placed in this directory, in a subdirectory with the name of the source client id, and with a filename of the remainder of the source URI. e.g. with a destination directory of **/tmp/audio-files** registered against an id of **audio**, a file with a source id of **example:client:12345#audio/cuts/a_file** will be placed in **/tmp/audio-files/example:client:12345/cuts/a_file**.

```

def example(transport: Transport):
    transport.file_transfer_service.register_destination('audio', '/tmp/audio-files')
    ...
    transport.file_transfer_service.deregister_destination('audio')

```

Unsolicited transfers may also be accepted by supplying a default directory for received files to be placed in. This has the same effect as registering an empty id as a destination.

```

def example(transport: Transport):
    transport.file_transfer_service.default_destination = '/tmp'
    ...

```



```
del transport.file_transfer_service.default_destination
```

If no default directory has been specified, then any unsolicited incoming transfers will be rejected.

Static Destinations

Rather than writing files to the filesystem, a `StaticDestinationFileFactory` can be registered as a destination rather than a directory or file path. This enables in-memory destinations to be accessed. The received data is stored in the factory as a nested dictionary, indexed by the source client id, and the remainder of the path.

```
def example(transport: Transport):
    factory = StaticDestinationFileFactory()
    client.file_transfer_service.register_source('static_data', factory)
    ...
    data = await factory['example:client:12345']['hello/world']
```

Custom Destinations

Other custom destination factories can also be created by implementing the `DestinationFileFactory` protocol. The `call` function on this class must take an `item_id` parameter and return an object conforming to the `DestinationFile` protocol.

```
class CustomDestination(DestinationFile):
    def __init__(self):
        self._done = asyncio.Event()

    async def __aenter__(self) -> Self:
        return self

    async def __aexit__(self, exc_t, exc_v, exc_tb):
        self._done.set()

    async def write(self, data: bytes):
        print(f"Received {len(data)} bytes")

    def __await__(self) -> None:
        return self._done.__await__()

class CustomDestinationFactory(SourceFileFactory[CustomDestination]):
    def __call__(self, item_id: str = "") -> CustomDestination|None:
        return CustomDestination()
```

4.3. Streams

A Transport client is able to broker streaming of RTP or other UDP data between two other clients, or between another client and itself. We will define three roles partaking in a file transfer, the

'Actor' which instigates the transfer, the 'Source' which provides the file to be transferred, and the 'Destination' which receives the file. The Transport instance can fulfill all these roles. The service handling these requests is accessed via the `stream_service` attribute.

4.3.1. Tasker

As the tasker, the `SilkwaveClient` can request for a UDP stream to be set up between two clients. The source and destination ids of the stream should be communicated to the application by some other means. A `StreamTask` object is returned from each request, and status updates are provided from the `subscribe_status` generator on the transfer. The status generator will exit when the stream is completed.

```
async def example(transport: Transport):
    stream = await transport.stream_service.setup_stream(
        source='example:client:12345#a_stream',
        destination=['example:client:54321#psd_display_1'])

    async def stop_stream():
        await asyncio.sleep(30)
        await stream.stop()

    asyncio.create_task(stop_stream())

    async for status in stream.subscribe_status():
        print(f"Stream status {status.description}")

    print("Stream stopped")
```

The client can also request a stream to started with data streaming to itself directly into a supplied `DestinationStream` object.

```
async def example(transport: Transport):
    stream = await transport.stream_service.request_stream(
        source='example:client:12345#a_stream',
        destination=NoneDestinationStream())

    async def stop_stream():
        await asyncio.sleep(30)
        await stream.stop()

    asyncio.create_task(stop_stream())

    async for status in stream.subscribe_status():
        print(f"Stream status {status.description}")

    print("Stream stopped")
```

4.3.2. Source Streams

Any available source streams must be registered with the stream service using `register_source`. This registers a factory against an id. This factory should return an instance of a `SourceStream` when supplied a stream id as a parameter.

The Transport expects streams to be transmitted as discrete packets, so a class implementing `SourceStream` should implement the `run` method and send packets using the `send_packet()` method.

When the stream is no longer required, the `run()` function will receive an `asyncio.CancelledError` exception, which should be allowed to propagate out of the function.

```
class HelloStream(SourceStream):
    def __init__(self):
        super().__init__(
            PacketIdentifierMask(
                offset=0,
                mask=0xffffffffffffffff,
            ),
        )

    async def run(self):
        while True:
            await self.send_packet(self.packet_identifier.to_bytes(8, 'big') + b'Hello World!')
            await asyncio.sleep(1)
```

RTP Source Streams

Streams are very often RTP streams, so an `RtpSourceStream` class is provided to handle creation of the `RtpHeader`, with just extension headers and payload left to the user. Subclasses of this only need to override the `run()` method, and make calls to `self.send_rtp_packet()`, passing the payload data, the timestamp delta to increase the timestamp from the previous packet and any extension headers.

```
class ExampleRtpSender(RtpSourceStream):
    def __init__(self):
        super().__init__(payload_type=999)

    async def run(self):
        while True:
            await self.send_rtp_packet(b"Hello, world!", 1)
            await asyncio.sleep(1)
```

There are a number of example RTP Source streams in the `pys.rfextgeo.streams.audio` and `pys.rfextgeo.streams.psd` modules of the `pys.rfextgeo` package.

4.3.3. Destination Streams

Any available destination streams must be registered with the stream service using

`register_destination`. This registers a factory against an id. This factory should return an instance of a `DestinationStream` when supplied a stream id as a parameter.

An implementation of a destination stream should implement the `run()` method, and receive packets from the 'packets()' asynchronous generator.

```
class ExampleReceiver(DestinationStream):

    async def run(self):
        async for packet in self.packets():
            print(f"Received {len(packet)} bytes.")
```

=== RTP Destination Streams

The `'RTPDestinationStream'` class provides an `'rtp_packets()'` async generator that yields a stream of RTP packet headers and payloads.

This class handles re-ordering of out-of-order packets, and any packets forwarded on are guaranteed to be in the correct order.

Obviously it cannot forward any packets dropped by UPD processing, so maintains a buffer (size specified by a constructor parameter) of out of order packets until the buffer is full.

Once the buffer is full, it gives up waiting for the lost packet and sends on any pending packets until the next gap.

As each packet is received, the header fields on the class are also set to the values for the header corresponding to the packet last yielded.

[source,python]

```
class ExampleRtpReceiver(RtpDestinationStream):
```

```
    async def run(self):
        async for header, payload in self.rtp_packets():
            print(f"Received payload of {len(payload)} bytes with payload type {header.payload_type}")
```

There are a number of example RTP Destination streams in the `pys.rfxextgeo.streams.audio` and `pys.rfxextgeo.streams.psd` modules of the `pys.rfxextgeo` package.